

Firmware Emulation Bằng Bộ Skill Tự Động

Firmware Research Team

May 21, 2026

Contents

1	Đặt vấn đề	2
2	Firmware emulation là gì?	2
3	Vì sao firmware emulation khó hơn tưởng tượng?	3
4	Vấn đề business: nếu làm thủ công thì sẽ gặp các vấn đề gì?	3
5	Ý tưởng của bộ skill	4
6	Cách dùng hiệu quả cho người mới	8
7	Kết quả thử nghiệm	8

1 Đặt vấn đề

Nếu một ngày mọi người nhận được một file firmware, chẳng hạn một file `.bin`, `.img`, `.raucb` hoặc `.img.gz`, cảm giác đầu tiên thường là: “Có file rồi, chạy lên thôi.” Nghe hợp lý. Nhưng firmware không giống một file `.exe` bình thường mà ta double-click rồi chờ cửa sổ hiện ra.

Firmware giống một chiếc hộp đóng kín chứa cả một phần của thế giới thiết bị bên trong. Ví dụ như, với router, nó có thể chứa hệ điều hành nhỏ, cấu hình mạng, web UI, daemon quản lý hệ thống, SSH server, firewall, package manager; với trạm sạc, nó có thể chứa service quản lý sạc, certificate, giao tiếp backend, CAN bus, OCPP, Modbus, quản lý certificate, database nội bộ, web management UI, v.v.

Nói cách khác, firmware không chỉ là “một chương trình”. Nó thường là một hệ sinh thái thu nhỏ.

Vấn đề là: từ một chiếc hộp như vậy, làm sao để dựng lại được một thiết bị ảo đủ thật để ta có thể thao tác? Không chỉ nhìn thấy log boot, mà còn vào được giao diện web, đăng nhập được, thao tác qua các chức năng quan trọng, và biết chắc rằng khi một route trả lỗi thì lỗi đó đến từ đâu.

Đó là lý do mình xây dựng bộ firmware emulation skill này.

2 Firmware emulation là gì?

Firmware emulation là việc tạo ra một môi trường giả lập để firmware tưởng rằng nó đang chạy trên phần cứng thật. Chẳng hạn như, nếu firmware vốn chạy trên một router MIPS, ta cần một máy ảo MIPS phù hợp; nếu firmware là x86, ta cần môi trường x86. Ngoài ra, còn cần rất nhiều thành phần tương ứng để đánh thức nó.

Một cách hình dung đơn giản, nếu ta muốn full-system emulate một thiết bị, ta cần dựng lại các thành phần chính:

- Ta cần một lớp phần cứng ảo, đóng vai CPU, RAM, ổ đĩa, card mạng, cổng serial, v.v. Ở đây, mình dùng **QEMU**, là một công cụ phổ biến và đủ tròn vai để làm điều này.
- **Kernel** - đóng vai trò đánh thức hệ thống. Để giả lập Kernel phải biết cách mount root filesystem, khởi động process đầu tiên và giao tiếp với driver.
- **Rootfs** - ổ đĩa chứa chương trình, thư viện, config, script init và các service.
- **Init system** - Manager đầu: service nào chạy trước, service nào chạy sau, log đi đâu.
- **Service** là các chương trình nền như web server, SSH, config daemon, database, message bus.

Có một điểm ban đầu mình đã bị nhầm là: boot được không đồng nghĩa với emulate thành công. Boot được chỉ có nghĩa là thiết bị ảo đã có tín hiệu có thể giao tiếp, nhưng rồi firmware vẫn sẽ nằm im.

Mình đã chia các trạng thái emulation thành nhiều tầng:

1. **Boot được**: kernel chạy, rootfs mount, init bắt đầu.
2. **Service lên**: các daemon cần thiết tồn tại và lắng nghe port đúng.
3. **Host truy cập được**: mở được service qua localhost.
4. **Feature dùng được**: sau authentication (nếu có), các route chức năng (không cần kết nối các phần cứng quá đặc thù) hoạt động, không có các lỗi như 500/502/404/400 v.v.

3 Vì sao firmware emulation khó hơn tưởng tượng?

Nếu nhìn sơ sơ, firmware emulation có vẻ là chuyện chọn đúng các thành phần mà tool hỗ trợ, sau đó sắp xếp lại theo đúng thứ tự, rồi run script là xong. Thực tế thì có vẻ đúng là như thế, nhưng với mình hay là những người mới làm quen về mảng firmware, mình hay vấp phải những vấn đề liên quan đến việc không hiểu rõ firmware đang cần gì, hoặc ta chọn sai thành phần nào đó. May mắn là AI có thể hỗ trợ ta rất mạnh trong việc debug những vấn đề đó, nhưng nếu không có skill để biết nên hỏi gì, thì ta sẽ dễ bị lạc trong một biển log và các option của QEMU.

Một vài vấn đề mình đã gặp phải khi emulate firmware bằng việc prompt thủ công:

- **AI debug chưa đủ sâu, dẫn đến chọn kiến trúc CPU sai, và phải chọn lại, dẫn đến dead loop rất mất thời gian:** firmware MIPS không thể chạy như x86. ARM, ARM64, MIPS big-endian, MIPS little-endian, x86_64 đều là các thế giới khác nhau.
- **Có rootfs nhưng thiếu kernel:** giống có ổ đĩa nhưng không có người khởi động máy tính.
- **Kernel boot được nhưng init không lên:** hệ thống tỉnh dậy rồi đứng yên.
- **Network không nối đúng:** guest có mạng riêng, host có mạng riêng, port forward mở nhưng service vẫn im lặng.
- **Chọn sai card mạng:** Chẳng hạn như thao tác cho QEMU cắm card `e1000`, nhưng firmware chỉ có driver `pcnet32`; kết quả là guest không biết nói chuyện với card mạng đó.
- **Thiếu các phần nhỏ như thiếu daemon, thiếu database hoặc thiếu config:** điều đó khiến cho việc, khi ta thao tác với các service, API phía sau luôn trả về lỗi.
- **Thiếu phần cứng thật:** Đây là vấn đề khó xử lý nhất. Có các phần cứng đặc thù của nhà sản xuất và đóng vai trò tối quan trọng trong việc vận hành firmware, nhưng ta lại không thể hoặc rất khó để giả lập nó như là các CAN bus, modem, TPM, NVRAM, serial device, storage partition, sensor hoặc service fieldbus, v.v.

Vì vậy, nếu như nói “firmware đã emulate được”, câu hỏi tiếp theo sẽ là: emulate được tới mức nào? Boot log? Shell? Web service? Login? Hay dùng được các chức năng gì?

4 Vấn đề business: nếu làm thủ công thì sẽ gặp các vấn đề gì?

Trong research, thời gian setup môi trường thường ta sẽ không thấy trong report, nhưng nó nuốt rất nhiều thời gian. Một firmware có thể mất vài giờ, thậm chí vài ngày, chỉ để tìm đúng cách boot. Với firmware có ít tài liệu tham khảo, thời gian này còn tăng lên vì mỗi bước đều phải đoán, thử, đọc log, đổi QEMU option, patch config, thử lại.

Vấn đề không chỉ là tốn thời gian. ngoài ra còn có ba vấn đề về mặt business lớn hơn:

- **Khó chuẩn hóa kết quả:** mỗi engineer có thể hiểu “emulate được” theo một cách khác nhau.
- **Khó reproduce:** nếu không ghi rõ kernel, QEMU command, patch, port mapping, route test, người khác khó chạy lại.
- **Khó audit:** khi kết luận một service hoạt động hoặc một route lỗi, cần biết bằng chứng đến từ runtime thật hay chỉ từ static analysis.

Bộ skill được thiết kế để ép quá trình đi theo một pipeline có bằng chứng rõ ràng: làm gì, thấy gì, sửa gì, còn thiếu gì, và thu thập kết quả cuối cùng là như thế nào, có phạm vi ra sao.

5 Ý tưởng của bộ skill

Ý tưởng nền tảng của bộ skill khá đơn giản: nói rằng một firmware đã được emulate thành công, cần biết firmware đó là gì, chạy trên kiến trúc nào, cần kernel hay rootfs nào, service nào phải lên, các API mà người dùng gọi để thao tác với các service có hoạt động hay chỉ trả lỗi, và nếu lỗi thì lỗi nằm ở phần nào, ví dụ như web server, reverse proxy, backend daemon, database, config, hay phần cứng không có trong môi trường lab, v.v.

Nói cách khác, bộ skill được sinh ra để biến một yêu cầu rất ngắn kiểu “hãy emulate firmware này” thành một nhóm thao tác, mỗi thao tác có đầu vào rõ ràng, đầu ra rõ ràng, và có artifact để các bước sau đọc lại.

Có bốn nguyên tắc thiết kế quan trọng.

1. **Tách bài toán lớn thành các vai trò nhỏ.** Firmware emulation là một bài toán nhiều lớp: format firmware, kiến trúc CPU, kernel, init, filesystem, network, service, UI, auth, backend, log, dependency và phần cứng giả lập. Nếu gom tất cả vào một skill duy nhất, workflow rất dễ trở thành một đoạn script dài khó kiểm soát. Bộ skill chia công việc thành các vai trò nhỏ để mỗi skill chỉ phải làm tốt phần của mình.
2. **Đi theo bằng chứng, không đi theo cảm giác.** Mỗi kết luận đều phải có artifact đi kèm: file nào đã đọc, service nào đã chạy, port nào đang listen, route nào đã test, lỗi nào đã thấy, log nào chứng minh điều đó. Nếu chưa có bằng chứng runtime thì chỉ được nói là giả thuyết, không được nói là thành công.
3. **Giữ phạm vi claim trung thực.** Emulate được tới đâu, claim tới đó. Ví dụ như, nếu chỉ chạy được userland service thì claim phải nói rõ đó là userland/service emulation, không được gọi nhầm là full-device emulation.
4. **Lấy khả năng sử dụng thật làm thước đo.** Đặt mục tiêu tối thượng là: Người dùng gần như phải có thể thao tác được với các chức năng trong firmware. Các lỗi không được bỏ qua nếu chúng nằm trên đường sử dụng chính.

Bộ skill hoạt động cơ bản như sau. Mỗi JSON artifact đều có phần metadata chung: ai tạo, tạo lúc nào, dựa trên input nào, có warning/error gì, độ nhạy dữ liệu ra sao và mức bằng chứng hiện tại. Để dễ đọc, phần **Đầu ra** bên dưới không liệt kê đầy đủ từng field, mà chỉ nêu các nhóm thông tin quan trọng nhất cần nhớ.

- **firmware-start**

- **Nhiệm vụ:** là cửa vào của toàn bộ workflow, dùng khi người dùng chỉ có một file firmware hoặc một thư mục đã extract và chưa biết phải gọi skill nào tiếp theo.
- **Đầu vào:** firmware path hoặc extracted directory, ghi chú authorization nếu có, gợi ý model/version nếu có, mục tiêu như full-system boot, service readiness, workload/API/IPC cần kiểm chứng, WBM/UI nếu firmware có giao diện, auth nếu firmware yêu cầu, và output workspace nếu người dùng chỉ định. Nói dễ hiểu, đầu vào là “ta có firmware nào, được phép làm gì với nó, và muốn đạt tới mức nào”.
- **Đầu ra:** `starter_request.json`, `next_skill_decision.json` và `blockers.json` nếu thiếu thông tin. Nói ngắn gọn, `starter_request.json` là phiếu yêu cầu ban đầu: firmware nào, được phép làm gì, mục tiêu là gì, workspace ở đâu và cần handoff sang skill nào. `next_skill_decision.json` nói bước tiếp theo nên chạy skill nào và vì sao. `blockers.json` ghi những thứ đang thiếu hoặc chưa an toàn để đi tiếp.
- **Mục tiêu:** biến một prompt ngắn thành yêu cầu có cấu trúc, giữ lại success gate ngay từ đầu, và làm rõ tiêu chí thành công theo loại firmware.

- **firmware-artifact-contract**

- **Nhiệm vụ:** đặt luật chung cho mọi artifact trong pipeline. Skill này không reverse engineering, không debug, không chạy firmware; nó kiểm tra cách ghi bằng chứng.
- **Đầu vào:** bất kỳ JSON artifact, JSONL observation stream, finding record, discovery artifact hoặc candidate report nào được các skill khác tạo ra.
- **Đầu ra:** kết quả kiểm tra artifact hợp lệ hay chưa, warning cần chú ý và blocker nếu artifact chưa đủ tin cậy. Với các file trạng thái như `emulation_success.json`, skill này giúp tách rõ từng gate: boot, service, host reachability, interface/API, auth nếu có, workload/feature và dependency.
- **Mục tiêu:** bảo đảm kết luận có provenance, có schema, có sensitivity label, có warnings/errors, và không gộp các trạng thái khác nhau thành một chữ “pass” mơ hồ. Boot được, service ready, host reachable, interface/API usable, auth handled, feature observed và dependency scoped là các mức khác nhau, phải ghi riêng theo đúng bối cảnh firmware.

- **firmware-intake-normalization**

- **Nhiệm vụ:** chuẩn hóa firmware trước khi phân tích sâu. Đây là bước trả lời các câu hỏi nền tảng: file này là gì, hash là gì, được extract bằng gì, rootfs nằm ở đâu, có những seed service hoặc binary nào đáng chú ý.
- **Đầu vào:** firmware artifact, extraction metadata và hashes. Đầu vào có thể là file raw ban đầu hoặc kết quả extract đã có sẵn, nhưng phải đủ thông tin để truy vết nguồn.
- **Đầu ra:** `firmware_manifest.json`, `extraction_summary.json` và binary seed list. Có thể hiểu `firmware_manifest.json` là “căn cước” của firmware: hash, format, model/version gọi ý, kiến trúc dự đoán và rootfs chính. `extraction_summary.json` là nhật ký extract: dùng tool nào, tìm thấy partition/filesystem gì, rootfs nằm đâu, có lỗi gì. Binary seed list là vài binary/service đáng xem trước.
- **Mục tiêu:** tạo căn cước cho firmware để các bước sau không làm việc trên một thư mục mơ hồ. Nếu chưa biết rootfs nào là chính, tool nào extract lỗi, hay firmware có nhiều partition chưa hiểu, skill này phải ghi blocker thay vì đoán.

- **firmware-binary-service-inventory**

- **Nhiệm vụ:** kiểm kê binary và service trong rootfs đã nhận diện. Nó tìm executable, script, interpreter, shared library, init path, daemon, quyền chạy, port gọi ý và service candidate.
- **Đầu vào:** `firmware_manifest.json` và rootfs candidate. Nói đơn giản, đây là lúc workflow đã biết “đồng file này là firmware nào” và bắt đầu hỏi “bên trong có chương trình nào có thể chạy”.
- **Đầu ra:** `binary_inventory.json`, `service_inventory.json` và `service_graph.json`. Ba file này trả lời ba câu hỏi: firmware có những chương trình nào, chương trình nào có vẻ là service cần chạy, và các service đó liên quan tới init script, config, port, library hoặc dependency nào.
- **Mục tiêu:** tạo bản đồ tĩnh ban đầu về các service có khả năng cần emulate, để workflow không phải mò từng file bằng cảm tính.

- **firmware-attack-surface-graph**

- **Nhiệm vụ:** nối service với input vector, parser, sink, privilege và auth boundary thành một graph có bằng chứng. Dù tên nghe thiên về security, trong bài toán emulation nó rất hữu ích vì giúp biết service nào thật sự quan trọng và đường tương tác nào cần kiểm chứng khi runtime hoạt động.
- **Đầu vào:** `binary_inventory.json`, `service_inventory.json` và `service_graph.json`.
- **Đầu ra:** `attack_surface_graph.json` và `target_selection.json`. File graph cho biết service nào nhận input từ đâu, đi qua parser hoặc boundary nào, và có bằng chứng mạnh hay yếu. File target selection chọn ra service/đường tương tác nên ưu tiên trước, kèm lý do.
- **Mục tiêu:** ưu tiên đúng mục tiêu: service nào expose ra network, service nào xử lý input từ API/IPC/CLI/update/config, service nào có quyền cao, service nào còn thiếu bằng chứng reachability.

- **firmware-input-path-discovery**

- **Nhiệm vụ:** tìm các đường input đi vào service đã chọn: HTTP route, CLI, IPC, file config, environment, update package, network message hoặc các đường điều khiển khác.
- **Đầu vào:** `target_selection.json`, `attack_surface_graph.json` và `selected binary/service`.
- **Đầu ra:** `input_vectors.json` và `input_path_chains.json`. File đầu tiên liệt kê các nguồn input có thể điều khiển được, ví dụ HTTP, IPC, CLI, config hoặc update package. File thứ hai mô tả input đó đi qua những bước nào trước khi chạm tới service, handler, parser hoặc hiệu ứng cần quan sát.
- **Mục tiêu:** hiểu service được điều khiển bằng cách nào để khi emulate ta biết nên tạo workload nào, message nào cần gửi, file/config nào cần chuẩn bị, endpoint nào cần gọi, và input nào chỉ là static guess chưa được runtime chứng minh.

- **firmware-discovery-orchestrator**

- **Nhiệm vụ:** điều phối toàn bộ pipeline theo bằng chứng hiện có. Nó không làm thay việc của các skill khác; nó đọc artifact, xác định còn thiếu gì, rồi chọn skill tiếp theo có khả năng tăng bằng chứng nhiều nhất.
- **Đầu vào:** artifact index, `firmware_manifest.json`, `hypothesis_ledger.json` và current evidence levels.
- **Đầu ra:** `discovery_plan.json`, `target_selection.json`, `hypothesis_ledger.json`, `next_skill_decision.json` và `blockers.json`. Nhìn thực dụng thì đây là bộ “bản đồ điều phối”: hiện đang biết gì, còn thiếu bằng chứng gì, giả thuyết nào đang mở, skill tiếp theo là gì, và blocker nào đang chặn tiến trình.
- **Mục tiêu:** giữ workflow không đi lạc. Nếu mục tiêu là full-system, full service, workload có thể dùng được, API/IPC có thể quan sát được, hoặc UI/auth khi firmware có lớp đó, orchestrator phải bảo toàn `emulation_success_gate` và tiếp tục route sang state discovery, service emulation, runtime observation hoặc debugging cho tới khi có đủ bằng chứng hoặc blocker rõ ràng.

- **firmware-service-state-discovery**

- **Nhiệm vụ:** lập bản đồ trạng thái, auth, dependency và reachability trước khi chạy sâu. Với mọi service, nó hỏi service cần file nào, config nào, daemon phụ nào, data directory nào, credential hoặc trạng thái khởi tạo nào, port/socket nào, và dependency phần cứng hoặc simulator nào. Nếu firmware có web UI thì bổ sung browser endpoint, static asset, API base URL, WebSocket, reverse proxy, login endpoint và session/token storage.
- **Đầu vào:** `service_inventory.json`, `attack_surface_graph.json` và `input_vectors.json`.
- **Đầu ra:** `service_state_map.json`, `reachability_blockers.json` và `ui_access_requirements.json`. `service_state_map.json` là bản đồ điều kiện để service chạy được: cần file nào, config nào, daemon phụ nào, port/socket nào, auth nào và dependency phần cứng/simulator nào. `reachability_blockers.json` ghi lý do chưa chạm được service. `ui_access_requirements.json` chỉ dùng khi firmware có giao diện, để ghi endpoint, API/WebSocket, reverse proxy và route chức năng.
- **Mục tiêu:** biết trước runtime cần điều kiện gì để service thật sự usable, từ database, certificate, message broker, storage partition, CAN/serial/GPIO/modem/TPM cho tới simulator. Đây là bước giúp mọi lỗi runtime có chỗ để truy nguyên, không riêng gì lỗi trên giao diện web.

- **firmware-service-emulation**

- **Nhiệm vụ:** dựng môi trường chạy cho service hoặc firmware lane đã chọn. Nó kiểm tra file, library, device node, port, env, init state, backend listener và dependency trước khi gọi là ready.

- **Đầu vào:** `service_inventory.json`, `service_state_map.json` và `hypothesis_ledger.json`.
- **Đầu ra:** `runtime_readiness.json`, `reachable_services.json`, `ui_access_status.json`, `emulation_success.json` hoặc `emulation_blocker.json`. Bộ file này cho biết runtime đã dựng bằng lane nào, service nào đã chạy, host có chạm được endpoint cần thiết không, UI/API nếu có đã ở trạng thái nào, và cuối cùng emulation pass hay fail ở gate nào. Nếu fail, blocker phải nói rõ gate nào hỏng và hướng debug tiếp.
- **Mục tiêu:** đưa firmware hoặc service lên một runtime có thể kiểm chứng được, ghi rõ lane là `qemu-system`, `qemu-user`, `chroot`, `container`, `proxy` hay `static-only`. Skill không gọi thành công nếu process chính sống nhưng dependency bắt buộc chết, host không truy cập được endpoint cần thiết, workload đại diện không chạy được, hoặc auth/interface chỉ mới hoạt động một phần nhưng chưa được scoped.

• firmware-runtime-observation

- **Nhiệm vụ:** quan sát hành vi thật khi runtime đã có thể chạy. Skill này dùng log, tracing, tcpdump, Frida, bpftrace, debugger-safe trace hoặc browser-level observation để chứng minh workload thật đã đi qua.
- **Đầu vào:** `runtime_readiness.json`, `hypothesis`, `workload` và `expected observation`. Workload có thể là gọi API, gửi IPC message, chạy CLI, nạp config, mô phỏng update package, mở một màn hình web, hoặc thực hiện một chức năng read-only sau auth nếu firmware yêu cầu.
- **Đầu ra:** `runtime_observation.json`, `ui_runtime_observation.json`, `observed_path_chains.json`, `observed_sink_hits.json` và `debug_transcript_index.json`. Các file này trả lời: đã chạy workload nào, kỳ vọng thấy gì, thực tế thấy gì, bằng chứng nằm ở log/trace nào, và đường input-to-effect nào đã được quan sát thật. Nếu có UI hoặc API, `ui_runtime_observation.json` ghi route/workload matrix ở mức đã redact.
- **Mục tiêu:** biến “tôi nghĩ nó chạy” thành “tôi đã quan sát đường chạy này”. Với firmware có giao diện hoặc auth, skill kiểm tra đúng đường người dùng thật sẽ đi và không ghi secret/token/cookie; với firmware headless, nó tập trung vào API, IPC, CLI, daemon state, packet flow hoặc log/runtime effect.

• firmware-debugging

- **Nhiệm vụ:** đào nguyên nhân khi có crash, lỗi runtime, sink hit hoặc câu hỏi root-cause cụ thể. Ví dụ: service chết lúc start, backend không mở port, endpoint trả 500, API 404, IPC không phản hồi, WebSocket không connect, binary đòi device không tồn tại, hoặc init script dừng giữa chừng.
- **Đầu vào:** crash hoặc suspicious effect, target process, `debug_plan.json` và `hypothesis`.
- **Đầu ra:** `debug_transcript_index.json`, crash-analysis handoff hoặc exploitability-modeling handoff nếu cần. File index này không phải dump debug thô, mà là bản tóm tắt: debug process nào, dùng cách attach/probe nào, thấy crash/effect gì, bằng chứng nằm ở đâu, có can thiệp runtime không, và root cause tạm thời là gì.
- **Mục tiêu:** tìm nguyên nhân có kiểm soát bằng GDB, gdbserver, QEMU gdbstub, guest breakpoint, host QEMU attach, log hoặc trace phù hợp; đồng thời ghi rõ thao tác nào có can thiệp runtime để người đọc không nhầm giữa hành vi firmware gốc và hành vi sau khi shim/patch.

• firmware-memory-layer

- **Nhiệm vụ:** biến kinh nghiệm đã được kiểm chứng thành pattern tái sử dụng cho các firmware sau. Skill này không được dùng để âm thầm sửa luật core hoặc lưu lại nhận định chưa chứng minh.
- **Đầu vào:** validated artifact, pattern draft, source evidence và artifact sensitivity label.

- **Đầu ra:** `memory_suggestions.json`, `memory_draft.md`, `promotion_decision.json` hoặc các pattern tái sử dụng. Nói đơn giản, đây là nơi ghi lại: pattern gì có thể dùng lại, bằng chứng nào chứng minh pattern đó, đã đủ điều kiện promote chưa, hay cần kiểm chứng thêm.
- **Mục tiêu:** giúp workflow ngày càng thông minh hơn sau mỗi case, nhưng vẫn giữ kỷ luật: pattern phải có ngày xác minh, nguồn bằng chứng, không chứa secret, và không thay thế việc kiểm chứng artifact hiện tại.

Luồng hoạt động có thể hình dung như sau: `firmware-start` ghi mục tiêu. `firmware-discovery-orchestrator` xem hiện tại đã có bằng chứng gì. Nếu thiếu nhận diện firmware thì route sang intake. Nếu thiếu danh sách service thì route sang inventory. Nếu chưa hiểu trạng thái chạy, auth, dependency hoặc workload thì route sang state discovery. Nếu có đủ điều kiện chạy thì route sang service emulation. Khi runtime đã lên, runtime observation kiểm tra bằng workload phù hợp: API, IPC, CLI, packet flow, log effect, hoặc browser path nếu firmware có giao diện. Nếu có lỗi, debugging quay lại tìm nguyên nhân. Sau mỗi vòng, artifact mới được ghi ra, orchestrator đọc lại, rồi quyết định tiếp. Vòng lặp dừng khi đạt success gate hoặc khi có blocker đủ rõ để nói chính xác vì sao chưa thể đạt.

6 Cách dùng hiệu quả cho người mới

Mỗi firmware ta sẽ sử dụng một one-shot prompt riêng. Nói rõ mục tiêu, không chỉ nói “boot firmware này”. Ngoài ra, nên thêm plugin **superpowers** và sử dụng thêm `/goal` để tối ưu hóa kết quả đầu ra, tránh việc phải prompt đi prompt lại nhiều lần.

Dùng `firmware-start` để one-shot full-system emulate firmware `/path/to/A.bin`, tạo script `start/stop/restart`, và ghi rõ blocker nếu không đạt.

7 Kết quả thử nghiệm

Trong kịch bản thử nghiệm mở rộng, bộ skill được chạy trên **12 mẫu firmware**: 10 mẫu OpenWrt thuộc nhiều target khác nhau và 2 mẫu **CHARX SEC-3100** ở hai phiên bản **1.7.1** và **1.9.0**. Điểm quan trọng ở đây không chỉ là số lượng mẫu, mà là độ đa dạng kiến trúc CPU và kiểu đóng gói firmware. Bộ skill không chỉ xử lý một bài toán “OpenWrt x86 dễ chạy”, mà phải đi qua nhiều thế giới khác nhau: full disk image, rootfs-only, sysupgrade image, SD card image và RAUC bundle của thiết bị công nghiệp.

Nhóm firmware	Số mẫu	Kiến trúc CPU	Loại input	Kết quả
OpenWrt x86_64 generic	1	x86_64	Full disk image	Full-system boot, service ready, host reachable, route matrix OK
OpenWrt ARM/ARM64 boards	6	ARMv7, ARM64/AArch64, Cortex-A9, Filogic/IPQ-class	Factory image, SD card image, sysupgrade image	Full-system boot thành công sau khi chọn đúng machine, NIC và storage profile
OpenWrt MIPS targets	3	MIPS little-endian và MIPS big-endian, bao gồm Malta BE	Sysupgrade image, rootfs-only image	Full-system boot thành công, xử lý khác biệt endian, kernel ngoài và NIC driver
CHARX SEC-3100	2	ARM Cortex-A7, i.MX6UL-class embedded platform	RAUC bundle, rootfs/kernel/DTB	Full-system boot qua kernel/init, service stack và WBM/API workload đạt success gate

Tổng hợp lại, bộ thử nghiệm bao phủ ít nhất bốn lớp kiến trúc lớn: **x86_64**, **ARMv7/ARM Cortex-A**, **ARM64/AArch64** và **MIPS** cả little-endian lẫn big-endian. Đây là điểm đáng giá nhất của bộ skill: workflow không bị khóa vào một CPU hay một dạng image duy nhất. Khi chuyển từ OpenWrt sang CHARX SEC-3100, bài toán cũng đổi từ router firmware sang firmware thiết bị công nghiệp có WBM, service phụ, dependency phân cứng và RAUC update bundle. Việc vẫn giữ được cùng một success gate cho các nhóm rất khác nhau này cho thấy bộ skill có thể dùng như một khung làm việc chung, không chỉ là một script QEMU viết riêng cho một model.